

Section A – Concept Application

Scenario : You are a junior developer at a fintech startup building a digital wallet app. The team is debating between Waterfall and Agile methodology. The project has frequently changing requirements and the client expects to review working features every two weeks.

Q1. Which development methodology would you recommend for this project, and why? Identify two specific risks the team would face if they chose the wrong methodology for this context.

Ans. **AGILE: Frequent Deliverables:** The client expects to review working features every two weeks. Agile is built around short iterations (typically 1-to-4-week sprints), making it perfectly aligned with delivering functional increments at this exact cadence.

Adaptability to Change: A fintech digital wallet in a startup environment inherently faces "frequently changing requirements." Agile welcomes shifting priorities, allowing the team to pivot and refine the product backlog without derailing the entire project timeline.

Risks of Choosing the Wrong Methodology (Waterfall)

If the team incorrectly chooses the **Waterfall** methodology for this context, they would face severe setbacks:

- ➔ **High Risk of Project Failure due to Scope Rigidity:** Waterfall relies on rigid, upfront documentation and sequential phases (Requirements → Design → Implementation). In an environment where requirements change frequently, sticking to a fixed plan means building features that may be obsolete by the time they are deployed, leading to wasted capital and a product that misses the market need.
- ➔ **Severe Client Dissatisfaction and Feedback Delays:** Since Waterfall typically only produces a working software build near the end of the entire lifecycle, the team would fail to meet the client's expectation of reviewing working features every two weeks. The client would remain in the dark for months, and any alignment issues or misunderstandings would only be discovered when it is too late (and too expensive) to fix.

Scenario : You are collaborating on a college project website with three teammates. Just before a client demo, one teammate pushed incomplete code directly to the main branch, breaking the layout for everyone else on the team.

Q2. Describe the Git workflow your team should adopt to prevent this from happening again. Name at least two Git commands your team should use as part of this workflow and explain the role of each command. Ans. **1. git checkout -b <branch-name>**

➔ **Role:** This command creates a brand-new feature branch and immediately switches your local workspace to it.

➔ **Why it helps:** It ensures that you are coding in an isolated environment. Even if your incomplete code breaks things locally, the main branch remains perfectly safe and stable for the rest of the team.

2. git pull origin main

-> **Role:** This fetches the latest, updated code from the remote repository's main branch and integrates it into your current local branch.

➔ **Why it helps:** Before submitting a Pull Request, running this command allows a developer to test how their new code interacts with the latest team updates. It helps catch layout conflicts locally *before* trying to merge anything online.

□

Alternative/Supplementary Command: git push origin <branch-name>

□ **Role:** Uploads the local feature branch to the remote repository.

➔ **Why it helps:** Instead of pushing to main, developers push their isolated branch to the cloud so they can safely open a Pull Request for peer review.

Scenario : You are building an online internship application form for a college placement portal. The form must collect: applicant name, email address, phone number, preferred domain (one of: Web, Data, Mobile, AI), and a resume file upload. The institution requires that no field be left empty and that the email field only accepts valid email formats — without writing any JavaScript.

- Q3. List the appropriate HTML input types you would use for each of the five fields. Explain how you would use HTML5 built-in validation attributes to enforce the two constraints mentioned, and why these attributes alone are not sufficient for production-level validation.

Ans.

Applicant Name: `<input type="text">` — Ideal for standard alphanumeric text strings.

`<input type="text" name="applicant_name" required>`

Email Address: `<input type="email">` — Automatically optimizes mobile keyboards for email entry (adds @ and .com shortcuts) and enables basic format checking.

`<input type="email" name="email_address" required>`

Phone Number: `<input type="tel">` — Opens a numeric keypad on mobile devices for easy phone number entry.

Preferred Domain (Web, Data, Mobile, AI): `<select>` with `<option>` tags, or a series of `<input type="radio">` buttons. Given it's a fixed list of four choices where they must pick exactly one, a dropdown list or radio button group restricts input perfectly.

Resume File Upload: `<input type="file">` — Allows users to browse and upload files from their local storage.

Scenario : You are designing a news portal that displays articles in a 3-column grid on desktop screens and in a single-column layout on mobile devices. Your team wants to use Bootstrap's grid system to achieve this without writing any custom media query CSS.

- Q4. Explain how Bootstrap's grid class system works to produce this responsive layout. Write the Bootstrap class combination you would apply to the article column div, and explain what each class in the combination controls.

Ans. Bootstrap's grid system uses a series of containers, rows, and columns to layout and align content. It is built with Flexbox and operates on a 12-column architecture. The system is mobile-first, meaning any layout styles applied apply to small screens

by default and scale *up* through defined responsive breakpoints (like md for medium, lg for large, etc.) using built-in CSS media queries behind the scenes.

If no large-screen overrides are specified, columns will automatically expand to fill the full width (12 columns), resulting in a stacked, single-column layout on mobile.

```
<div class="col-12 col-lg-4">  
  <!-- Article Content Goes Here -->  
</div>
```

Scenario : You are writing a C program to record daily rainfall data for a month (30 days). You need to store all 30 readings, calculate the monthly average, and then identify which specific days recorded rainfall above that average.

Q5. Explain why an array is the correct data structure for this task compared to using 30 separate variables. Describe the two-step logic your program would follow: first to compute the average, then to identify the above-average days — and explain why both steps cannot be done in a single loop.

Ans.

Using **30 separate variables** (e.g., day1, day2, ..., day30) would make the code incredibly redundant, hard to maintain, and inefficient. An **array** is the ideal choice for several reasons:

- ➔ **Code Simplicity and Scalability:** Instead of writing 30 individual lines of code to accept inputs and another 30 lines to sum them up, an array lets you group all data under a single name (float rainfall[30]). This allows you to manage everything cleanly using loops.
- ➔ **Dynamic Indexed Access:** Arrays allow you to access or manipulate elements using an index variable inside a loop (e.g., rainfall[i]). You cannot dynamically loop through separate variable names like day1 or day2 using a loop counter.

The Two-Step Logic

The program must be split cleanly into two independent phases:

Step 1: Compute the Average

1. Initialize a total_rainfall variable to 0.
2. Use a loop to collect all 30 daily readings from the user, adding each input value to total_rainfall.
3. After the loop finishes, divide total_rainfall by 30 to compute the final average_rainfall.

Step 2: Identify Above-Average Days

1. Start a completely new loop from index 0 to 29.
2. Inside this loop, evaluate each element: if (rainfall[i] > average_rainfall).
3. If the condition is met, print the day number (\$i + 1\$) and its corresponding value.

Single Loop

You **cannot** check if a day's rainfall is above average in the exact same loop where you are collecting data and calculating the sum because of a fundamental math dependency:

The Dependency Problem: You cannot know if a single day's rainfall is "above average" until you know what the final average actually is. The true average cannot be determined until the very last value (Day 30) is entered and added to the total. If you tried to perform the check in the first loop, checking Day 2 against a running total or a partial average would yield inaccurate comparisons and incorrect results. Therefore, the first loop must finish entirely to establish the baseline average before a second loop can begin evaluating individual days against it.

Scenario : You are debugging a C program that uses a pointer to traverse a string entered by the user. The program works correctly when the user types a name, but crashes immediately when the input field is left empty (the user just presses Enter).

Q6. : Explain why the program crashes on an empty string input and describe the check your pointer-based solution must perform before dereferencing the pointer. How does traversing a string using a pointer differ from traversing it using array index notation, and which approach makes the empty-input bug easier to catch?

Ans.

When a user simply presses Enter on an empty input field, functions like `fgets()` or standard buffer readers still write data to the memory buffer. They store a **null terminator ('\0')** at the very beginning of the string (index 0) to mark it as empty (or a newline '\n' followed by '\0').

The program crashes because of how the traversal loop is structured. If the code blindly attempts to move or inspect memory *beyond* that initial null terminator, or if

the pointer is initialized to a NULL memory address because the read operation failed completely, **dereferencing** it (trying to read `*ptr`) causes an invalid memory access violation, resulting in an immediate **Segmentation Fault (Crash)**.

2. The Essential Check Before Dereferencing

To safeguard a pointer-based solution, the code must perform a validation check to ensure the pointer is neither pointing to a NULL memory space nor standing at the immediate end of a string before reading its contents:

```
if (ptr != NULL && *ptr != '\0') {  
  
}  

```

3. Pointer Traversal vs. Array Index Notation

The choice between these two styles changes how memory addresses are incremented and handled in C: **Pointer Traversal**

➔ **How it works:** The memory address itself is variable. You maintain a pointer tracking a specific address in memory and continuously move it forward to the next byte using address arithmetic (e.g., `ptr++`).

➔ **Dereferencing:** Checked via raw content value inspection (`*ptr`).

Array Index Notation

➔ **How it works:** The base memory address remains completely stationary. Instead of moving the pointer, you track an independent integer counter variable (e.g., `i = 0`), which acts as an offset from the base address.

➔ **Dereferencing:** Checked via bracket indices (`string[i]`).

Why Array Notation Wins for Safety:

1. **Explicit Bounds Control:** With array notation, your loop conditions almost always look like `while (i < length)`. This forces you to explicitly think about the current size/length of the string upfront. If the input is empty, `length` evaluates to 0, the loop condition (`0 < 0`) fails immediately, and the code safely skips traversal entirely.
2. **Less Risk of Wild Pointers:** Pointer arithmetic requires you to precisely track memory address boundaries. If a pointer overshoots a null terminator, it starts reading raw, unallocated RAM. Array notation keeps the underlying pointers anchored safely, making it much harder to accidentally step out of bounds.

